

第 6 章-深度神经网络 – 第 6.5 节-混合密度网络

《深度学习基础》课程小组

2026 年 4 月 26 日

目录

1	Introduction	1
2	6.5.1 Robot kinematics example	1
3	6.5.2 Conditional mixture distributions	3
4	6.5.3 Gradient optimization	6
5	6.5.4 Predictive distribution	7

1 Introduction

So far in this chapter we have discussed neural networks whose outputs represent simple probability distributions comprising either a Gaussian for continuous variables or a binary distribution for discrete variables.

We close the chapter by showing how a neural network can represent more general conditional probabilities by treating the outputs of the network as the parameters of a more complex distribution, in this case a Gaussian mixture model.

This is known as a *mixture density network*, and we will see how to define the associated error function and the corresponding output-unit activation functions.

混合密度网络输出目标条件分布的参数，而不是目标变量本身。

2 6.5.1 Robot kinematics example

The goal of supervised learning is to model a conditional distribution $p(\mathbf{t}|\mathbf{x})$, which for many simple regression problems is chosen to be Gaussian.

However, practical machine learning problems can often have significantly non-Gaussian distributions.

These can arise, for example, with *inverse problems* in which the distribution can be multimodal, in which case the Gaussian assumption can lead to very poor predictions.

As a simple illustration of an inverse problem, consider the kinematics of a robot arm, as illustrated in Figure 6.16.

The *forward problem* involves finding the end-effector position given the joint angles and has a unique solution.

However, in practice we wish to move the end effector of the robot to a specific position, and to do this we must set appropriate joint angles.

We therefore need to solve the inverse problem, which has two solutions, as seen in Figure 6.16.

Forward problems often correspond to causality in a physical system and generally have a unique solution.

For instance, a specific pattern of symptoms in the human body may be caused by the presence of a particular disease.

In machine learning, however, we typically have to solve an inverse problem, such as trying to predict the presence of a disease given a set of symptoms.

If the forward problem involves a many-to-one mapping, then the inverse problem will have multiple solutions.

For instance, several different diseases may result in the same symptoms.

In the robotics example, the kinematics is defined by geometrical equations, and the multimodality is readily apparent.

However, in many machine learning problems the presence of multimodality, particularly in problems involving spaces of high dimensionality, can be less obvious.

For tutorial purposes, however, we will consider a simple toy problem for which we can easily visualize the multimodality.

The data for this problem is generated by sampling a variable x uniformly over the interval $(0, 1)$,

to give a set of values $\{x_n\}$, and the corresponding target values t_n are obtained by computing the function $x_n + 0.3 \sin(2\pi x_n)$ and then adding uniform noise over the interval $(-0.1, 0.1)$.

The inverse problem is then obtained by keeping the same data points but exchanging the roles of x and t .

Figure 6.17 shows the data sets for the forward and inverse problems, along with the results of fitting two-layer neural networks having six hidden units and a single linear output unit by minimizing a sum-of-squares error function.

Least squares corresponds to maximum likelihood under a Gaussian assumption.

We see that this leads to a good model for the forward problem but a very poor model for the highly non-Gaussian inverse problem.

练习：复现这个例子：

首先生成数据 (x_n, y_n) ，然后通过一个两层神经网络（6 个隐藏层）拟合 $y = f(x)$ 。

再通过同样的网络拟合 $x = g(y)$ 。

问题：数据集的多模态性指的是什么？

3 6.5.2 Conditional mixture distributions

We therefore seek a general framework for modelling conditional probability distributions.

This can be achieved by using a mixture model for $p(\mathbf{t}|\mathbf{x})$ in which both the mixing coefficients as well as the component densities are flexible functions of the input vector $\mathbf{x}$, giving rise to a *mixture density network*.

For any given value of $\mathbf{x}$, the mixture model provides a general formalism for modelling an arbitrary conditional density function $p(\mathbf{t}|\mathbf{x})$.

Provided we consider a sufficiently flexible network, we then have a framework for approximating arbitrary conditional distributions.

Here we will develop the model explicitly for Gaussian components, so that

$$p(\mathbf{t}|\mathbf{x}) = \sum_{k=1}^K \pi_k(\mathbf{x}) \mathcal{N}(\mathbf{t} | \boldsymbol{\mu}_k(\mathbf{x}), \sigma_k^2(\mathbf{x})) . \quad (6.38)$$

This is an example of a *heteroscedastic* model in which the noise variance on the data is a function of the input vector $\mathbf{x}$.

Instead of Gaussians, we can use other distributions for the components, such as Bernoulli distributions if the target variables are binary rather than continuous.

We have also specialized to the case of isotropic covariances for the components, although the mixture density network can readily be extended to allow for general covariance matrices by representing the covariances using a Cholesky factorization (Williams, 1996).

在混合密度网络中，为了简化，假设每个高斯分量的协方差矩阵是“各向同性”的（isotropic）。这意味着对于单个高斯分量，其协方差矩阵是一个数量矩阵。从几何上看，这表示每个高斯分量在所有维度上具有相同的方差，并且变量之间的协方差为零。它产生的概率密度轮廓是圆形（二维）或球形（多维）的。

各向同性的简化假设可以扩展到允许每个高斯分量拥有“一般协方差矩阵”。Cholesky 分解是一种将任意正定矩阵（协方差矩阵必须是正定的）分解为一个下三角矩阵及其转置乘积的方法（ $\Sigma = \mathbf{L}\mathbf{L}^T$ ）。神经网络可以学习下三角矩阵 $\mathbf{L}$ 的元素，然后通过 $\mathbf{L}\mathbf{L}^T$ 得到所需的协方差矩阵。

Williams 在 1996 年的文献详细介绍了如何利用 Cholesky 分解来实现这种扩展。

Even with isotropic components, the conditional distribution $p(\mathbf{t}|\mathbf{x})$ does not assume factorization with respect to the components of $\mathbf{t}$ (in contrast to the standard sum-of-squares regression model) as a consequence of the mixture distribution.

即使混合密度网络中的每个高斯分量都假设为各向同性的（即方差在各个维度上相同），整个条件概率分布 $p(\mathbf{t}|\mathbf{x})$ 也不会将目标变量 $\mathbf{t}$ 的各个分量视为相互独立的。

这与标准的最小二乘回归模型不同，后者通常假设误差在各个输出维度上是独立的，而混合分布的特性使得即使单个分量内部独立，整体分布也能捕捉到 $\mathbf{t}$ 不同分量间的依赖关系。

We now take the various parameters of the mixture model, namely the mixing coefficients $\pi_k(\mathbf{x})$, the means $\mu_k(\mathbf{x})$, and the variances $\sigma_k^2(\mathbf{x})$, to be governed by the outputs of a neural network that takes $\mathbf{x}$ as its input.

The structure of this mixture density network is illustrated in Figure 6.18.

The mixture density network is closely related to the mixture-of-experts model (Jacobs et al., 1991).

The principal difference is that a mixture of experts has independent parameters for each component model in the mixture, whereas in a mixture density network, the same function is used to predict the parameters of all the component densities as well as the mixing coefficients, and so the nonlinear hidden units are shared amongst the input-dependent functions.

The neural network in Figure 6.18 can, for example, be a two-layer network having sigmoidal (tanh) hidden units.

If there are K components in the mixture model (6.38), and if it has L components, then the network will have K output units pre-activations denoted by a_k^π that determine the mixing coefficients $\pi_k(\mathbf{x})$, K outputs denoted by a_k^μ that determine the Gaussian standard deviations $\sigma_k(\mathbf{x})$, and $K \times L$ outputs denoted by a_{kj}^μ that determine the components $\mu_{kj}(\mathbf{x})$ of the Gaussian means $\boldsymbol{\mu}_k(\mathbf{x})$.

The total number of network outputs is given by $(L + 2)K$, while the usual L outputs for a network that simply predicts the conditional means of the target variables.

混合模型由 K 个 L 维正态分布叠加得到。这样共有 LK 个均值参数。

还有 K 个数量矩阵（协方差），一共 K 个参数。

还有 K 个混合系数。因此混合密度网络一共有 $(L + 2)K$ 个输出。

The mixing coefficients must satisfy the constraints

$$\sum_{k=1}^K \pi_k(\mathbf{x}) = 1, \quad 0 \leq \pi_k(\mathbf{x}) \leq 1, \quad (6.39)$$

which can be achieved using a set of softmax outputs:

$$\pi_k(\mathbf{x}) = \frac{\exp(a_k^\pi)}{\sum_{l=1}^K \exp(a_l^\pi)}. \quad (6.40)$$

Similarly, the variances must satisfy $\sigma_k^2(\mathbf{x}) \geq 0$ and so can be represented in terms of the exponentials of the corresponding network pre-activations using

$$\sigma_k(\mathbf{x}) = \exp(a_k^\sigma). \quad (6.41)$$

Finally, because the means $\boldsymbol{\mu}_k(\mathbf{x})$ have real components, they can be represented directly by the network outputs:

$$\mu_{kj}(\mathbf{x}) = a_{kj}^\mu \quad (6.42)$$

in which the output-unit activation functions are given by the identity $f(a) = a$.

The learnable parameters of the mixture density network comprise the vector $\mathbf{w}$ of weights and biases in the neural network, which can be set by maximum likelihood or equivalently by minimizing an error function defined to be the negative logarithm of the likelihood.

For independent data, this error function takes the form

$$E(\mathbf{w}) = - \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k(\mathbf{x}_n, \mathbf{w}) \mathcal{N}(\mathbf{t}_n | \boldsymbol{\mu}_k(\mathbf{x}_n, \mathbf{w}), \sigma_k^2(\mathbf{x}_n, \mathbf{w})) \right\} \quad (6.43)$$

where we have made the dependencies on $\mathbf{w}$ explicit.

误差函数定义为似然函数的负对数。

4 6.5.3 Gradient optimization

To minimize the error function, we need to calculate the derivatives of the error $E(\mathbf{w})$ with respect to the components of $\mathbf{w}$.

We will see later how to compute these derivatives automatically.

第 8.2 节：自动微分。

It is instructive, however, to derive suitable expressions for the derivatives of the error with respect to the output-unit pre-activations explicitly as this highlights the probabilistic interpretation of these quantities.

误差函数相对于输出单元（混合密度的参数）的预激活值（在激活函数之前的值）的导数表达式。

Because the error function (6.43) is composed of a sum of terms, one for each training data point, we can consider the derivatives for a particular input vector $\mathbf{x}_n$ with associated target vector $\mathbf{t}_n$.

单个（输入与输出）数据点的误差，对混合密度参数的预激活值的导数。

The derivatives of the total error E are obtained by summing over all data points, or the individual gradients for each data point can be used directly in gradient-based optimization algorithms.

在随机梯度下降这类基于梯度的优化算法里，可以直接使用单个数据点（或一个小批量数据点）计算出的梯度来更新参数。

It is convenient to introduce the following variables:

$$\gamma_{nk} = \gamma_k(\mathbf{t}_n | \mathbf{x}_n) = \frac{\pi_k \mathcal{N}_{nk}}{\sum_{l=1}^K \pi_l \mathcal{N}_{nl}} \quad (6.44)$$

where $\mathcal{N}_{nk}$ denotes $\mathcal{N}(\mathbf{t}_n | \boldsymbol{\mu}_k(\mathbf{x}_n), \sigma_k^2(\mathbf{x}_n))$.

These quantities have a natural interpretation as posterior probabilities for the components of the mixture in which the mixing coefficients $\pi_k(\mathbf{x})$ are viewed as $\mathbf{x}$ -dependent prior probabilities.

The derivatives of the error function with respect to the network output pre-activations governing the mixing coefficients are given by

$$\frac{\partial E_n}{\partial a_k^\pi} = \pi_k - \gamma_{nk}. \quad (6.45)$$

使用求导的链式法则，固定 n, k ，分别计算 E_n 对 π_m 的导数、 π_m 对 a_k^π 的导数。然后对 m 求和。

Similarly, the derivatives with respect to the output pre-activations controlling the component means are given by

$$\frac{\partial E_n}{\partial a_{kl}^\mu} = \gamma_{nk} \left\{ \frac{\mu_{kl} - t_{nl}}{\sigma_k^2} \right\}. \quad (6.46)$$

Finally, the derivatives with respect to the output pre-activations controlling the component variances are given by

$$\frac{\partial E_n}{\partial a_k^\sigma} = \gamma_{nk} \left\{ L - \frac{\|\mathbf{t}_n - \boldsymbol{\mu}_k\|^2}{\sigma_k^2} \right\}. \quad (6.47)$$

5 6.5.4 Predictive distribution

We illustrate the use of a mixture density network by returning to the toy example of an inverse problem shown in Figure 6.17.

Plots of the mixing coefficients $\pi_k(x)$, the means $\mu_k(x)$, and the conditional density contours corresponding to $p(t|x)$, are shown in Figure 6.19.

The outputs of the neural network, and hence the parameters in the mixture model, are necessarily continuous single-valued functions of the input variables.

However, we see from Figure 6.19(c) that the model is able to produce a conditional density that is unimodal for some values of x and trimodal for other values by modulating the amplitudes of the

mixing components $\pi_k(\mathbf{x})$.

Once a mixture density network has been trained, it can predict the conditional density function of the target data for any given value of the input vector.

This conditional density represents a complete description of the generator of the data, so far as the problem of predicting the value of the output vector is concerned.

From this density function, we can calculate more specific quantities that may be of interest in different applications.

One of the simplest of these is the mean, corresponding to the conditional average of the target data, and is given by

$$\mathbb{E}[\mathbf{t}|\mathbf{x}] = \int \mathbf{t} p(\mathbf{t}|\mathbf{x}) d\mathbf{t} = \sum_{k=1}^K \pi_k(\mathbf{x}) \boldsymbol{\mu}_k(\mathbf{x}) \quad (6.48)$$

where we have used (6.38).

Because a standard network trained by least squares approximates the conditional mean, we see that a mixture density network can reproduce the conventional least-squares result as a special case.

一个用最小二乘法训练的标准神经网络，其输出近似于目标变量 $\mathbf{t}$ 在给定输入 $\mathbf{x}$ 下的条件期望（即条件均值） $\mathbb{E}[\mathbf{t}|\mathbf{x}]$ 。从公式 (6.48) 可以看出，混合密度网络（MDN）预测的条件均值是各分量均值的加权平均。如果 MDN 只学习一个分量（ $K = 1$ ），并且这个分量的方差被忽略或固定，那么 MDN 输出的均值 $\boldsymbol{\mu}_1(\mathbf{x})$ 就退化为标准网络的输出。因此，标准的最小二乘回归结果可以看作是混合密度网络的一种特殊情况。

Of course, as we have already noted, for a multimodal distribution the conditional mean is of limited value.

We have similarly evaluate the variance of the density function about the conditional average, to give

$$\begin{aligned} s^2(\mathbf{x}) &\equiv \mathbb{E}[\|\mathbf{t} - \mathbb{E}[\mathbf{t}|\mathbf{x}]\|^2 | \mathbf{x}] \\ &= \sum_{k=1}^K \pi_k(\mathbf{x}) \left\{ \sigma_k^2(\mathbf{x}) + \left\| \boldsymbol{\mu}_k(\mathbf{x}) - \sum_{l=1}^K \pi_l(\mathbf{x}) \boldsymbol{\mu}_l(\mathbf{x}) \right\|^2 \right\} \end{aligned} \quad (1) \quad (6.49-6.50)$$

where we have used (6.38) and (6.48).

这里计算了条件分布 $p(\mathbf{t}|\mathbf{x})$ 相对于其条件均值 $\mathbb{E}[\mathbf{t}|\mathbf{x}]$ 的方差 $s^2(\mathbf{x})$ 。公式 (6.49-6.50) 给出了这个方差的具体表达式，它不仅考虑了每个高斯分量自身的方差 $\sigma_k^2(\mathbf{x})$ ，还考虑了该分量的均值 $\boldsymbol{\mu}_k(\mathbf{x})$ 与总体条件均值之间的差异的平方。这反映了混合模型中方差的来源有两个：分量内部的不确定性以及分量之

间的离散程度。

This is more general than the corresponding least-squares result because the variance is a function of $\mathbf{x}$.

MDN 计算出的方差 $s^2(\mathbf{x})$ 是输入 $\mathbf{x}$ 的函数。这比标准的最小二乘回归更通用，因为在标准最小二乘中，通常假设误差（噪声）的方差是一个不随输入 $\mathbf{x}$ 变化的常数（即同方差，homoscedastic）。而 MDN 允许方差随 $\mathbf{x}$ 变化（异方差，heteroscedastic），能更好地建模数据中变化的不确定性。

We have seen that for multimodal distributions, the conditional mean can give a poor representation of the data.

再次强调，对于多峰分布，条件均值不能很好地表示数据。

For instance, in controlling the simple robot arm shown in Figure 6.16, we need to pick one of the two possible joint angle settings to achieve the desired end-effector location, but the average of the two solutions is not itself a solution.

以图 6.16 的机器人臂为例，给定末端执行器的目标位置，存在两个可行的关节角度组合。控制机器人时，必须选择其中一个解，而这两个解的平均值通常不是一个有效的解（即无法让末端到达目标位置）。

In such cases, the conditional mode may be of more value.

在这种情况下，条件众数（conditional mode，即概率分布中概率最高的那个或那些 $\mathbf{t}$ 值）可能比条件均值更有用。

Because the conditional mode for the mixture density network does not have a simple analytical solution, a numerical iteration is required.

混合密度网络的条件众数没有简单的解析解（不像高斯分布的众数就是均值那样直接），找到它需要通过数值迭代的方法来计算。

A simple alternative is to take the mean of the most probable component (i.e., the one with the largest mixing coefficient) at each value of $\mathbf{x}$.

一个简单的替代方案是：对于每一个输入 $\mathbf{x}$ ，找出此时混合模型中权重最大的那个分量（即 $\pi_k(\mathbf{x})$ 最大的那个 k ），然后取这个分量的均值 $\mu_k(\mathbf{x})$ 作为输出。这并不是严格的条件众数，但在很多情况下是一个实用的近似。

This is shown for the toy data set in Figure 6.19(d).

图 6.19(d) 展示了在示例数据集上，使用“最可能分量的均值（红点）”这一替代方案的结果。